Actividad 4. Patrones de creación y patrones estructurales

Diseño de la solución para “Los Atuendos”: software de administración de alquiler de vestuario

Christian Rey Mora López

Grupo: Christian Rey Mora López

Ingeniería de Software, Fundación Universitaria Compensar

Patrones de Diseño de Software

Profesor: John Fredy Céspedes Beltrán

31 de octubre de 2026

Introducción

En la actividad anterior entendimos que un patrón de diseño no es código terminado, es una solución probada a un problema que se repite. Ahora toca aplicarlo, el caso de “Los Atuendos” es un negocio de alquiler de vestidos de dama, trajes de caballero y disfraces que necesita registrar prendas, clientes, empleados y servicios de alquiler, hacer consultas y manejar una lista de prendas para lavandería con prioridades. El diagrama de clases del enunciado resuelve el modelo de datos, pero no dice cómo se crean los objetos, cómo se conectan las capas ni cómo crece el sistema cuando aparezca un tipo nuevo de prenda o un proveedor nuevo de lavado.

En este documento relaciono los patrones de creación y estructurales que elegí, justifico cada uno con un problema concreto del caso, reconstruyo el diagrama de clases con los patrones incorporados y presento el diagrama de secuencia de cada patrón. Como el grupo está conformado solo por mí, la fase individual y la unificada coinciden; la retroalimentación la hice como una revisión propia del diseño antes de implementarlo en Java con NetBeans (Debrauwer, 2018; Gamma et al., 2003).

Análisis del diagrama de clases propuesto

Al revisar el diagrama del enunciado, que tiene una clase Persona con Cliente y Empleado, una clase Prenda con VestidoDama, TrajeCaballero y Disfraz, y la clase ServicioAlquiler, encontré seis puntos donde el diseño se vuelve rígido si se programa tal cual:

- La creación de prendas depende del tipo, si la vista hace un *if* por cada tipo, agregar uno nuevo obliga a tocar la interfaz, la base de datos y las validaciones al mismo tiempo.
- Un servicio de alquiler solo es válido si tiene número, cliente, empleado, al menos una prenda y una fecha; un constructor con seis parámetros es fácil de llamar mal.
- Toda la información debe ir a una base de datos relacional, y abrir una conexión por consulta desperdicia recursos y deja el esquema sin un único responsable.
- La interfaz necesitaría conocer a todos los DAO, las reglas de disponibilidad y la lista de lavandería.

- Una prenda devuelta puede llegar manchada, ser delicada o tener prioridad del administrador, y esas condiciones se combinan entre sí.
- El envío a lavandería depende de un proveedor externo cuya interfaz no controlamos, y los reportes se muestran en pantalla y también como notificación de texto.

Patrones identificados y justificación

La Tabla 1 relaciona cada patrón con el problema del caso que resuelve. Elegí tres patrones de creación y los cinco estructurales de la unidad, porque cada uno tiene un lugar claro en el negocio y no los puse por cumplir; si un patrón no resuelve nada, termina siendo un antipatrón, como vimos en la actividad 1.

Tabla 1. *Primera fase del trabajo individual*

N.º	Patrón	Tipo	Problema del caso	Aplicación en el diseño	Justificación
1	Singleton	Creación	Una sola conexión JDBC y un solo responsable de crear el esquema de la base de datos.	ConexionBD con constructor privado y getInstancia(); todos los DAO la piden a través de DaoBase.	Evita abrir conexiones por cada consulta y garantiza que las tablas se creen una sola vez, desde un punto de acceso global controlado.
2	Factory Method	Creación	Cada tipo de prenda tiene atributos y validaciones propias.	FabricaPrenda (creador abstracto) con crearPrenda(); FabricaVestidoDama, FabricaTrajeCaballero y FabricaDisfraz deciden la clase concreta.	La fachada y el DAO piden “una prenda de este tipo” sin conocer la subclase; un tipo nuevo es una fábrica nueva, sin tocar el resto.
3	Builder	Creación	El servicio de alquiler se arma por pasos: cliente, empleado, varias prendas, fechas y número consecutivo.	ServicioAlquiler.Builder con métodos encadenados y build() que verifica que el servicio esté completo.	Separa la construcción de la representación y impide que exista un servicio sin cliente, empleado o prendas.
4	Composite	Estructural	Un cliente alquila varias prendas y la consulta por talla debe separar las prendas por tipo.	ComponenteAlquiler (componente), Prenda (hoja) y GrupoPrendas (compuesto).	El valor total del servicio y el árbol talla → tipo → prenda se recorren igual, sin preguntar si el elemento es una prenda o un grupo.
5	Facade	Estructural	La interfaz gráfica tendría que coordinar cinco DAO, fábricas, validaciones y la lavandería.	LosAtuendosFacade ofrece una operación por cada funcionalidad del enunciado.	Reduce el acoplamiento: la vista solo conoce la fachada y las reglas del negocio quedan en un único lugar.
6	Decorator	Estructural	Las condiciones de lavado (manchada, delicada, prioridad del administrador) se combinan.	ItemLavanderia, PrendaEnLavanderia y los decoradores PrendaManchada,	Se agregan responsabilidades en tiempo de ejecución sin crear una subclase por cada combinación;

				PrendaDelicada y PrioridadAdministrador.	cualquier decorador da prelación en el envío.
7	Adapter	Estructural	El proveedor de lavandería recibe arreglos de referencias y devuelve un número de tanda.	ServicioLavanderia (objetivo), LavanderiaExterna (adaptado) y AdaptadorLavanderiaExterna.	La aplicación trabaja con su propia interfaz; si cambia el proveedor, solo cambia el adaptador.
8	Bridge	Estructural	Los mismos reportes se muestran como HTML en pantalla y como texto plano en la notificación de envío.	Reporte y sus subclases (abstracción) con FormatoSalida, FormatoHtml y FormatoTexto (implementación).	Reportes y formatos crecen por separado: 5 reportes × 2 formatos sin 10 clases combinadas.

Fuente: Elaboración propia

Los patrones no quedan sueltos, se apoyan entre sí. La fachada usa el Factory Method para registrar prendas y el Builder para registrar servicios; el Builder guarda las prendas en un GrupoPrendas, que es el Composite; el DAO de lavandería reconstruye cada ítem con los decoradores que correspondan, y la fachada entrega la tanda al Adapter. Al final, los resultados se muestran con el Bridge. Esa cadena es la que le da cohesión al diseño.

Diagrama de clases con los patrones incorporados

Reconstruí el diagrama del enunciado en Mermaid y lo dividí en dos vistas para que se pueda leer. La primera conserva el modelo del negocio (Persona, Cliente, Empleado, Prenda y sus subtipos, ServicioAlquiler) y le agrega los patrones de creación y el Composite; la segunda muestra la capa de servicios con los patrones estructurales. El código fuente de los diagramas está en el Anexo A.

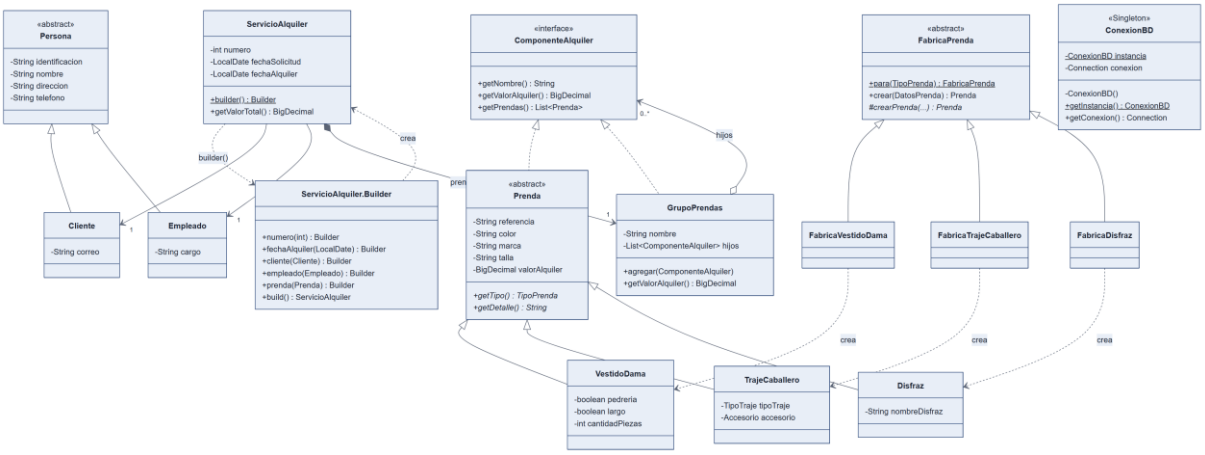


Ilustración 1. *Diagrama de clases del modelo de dominio con Singleton, Factory Method, Builder y Composite. Elaboración propia.*

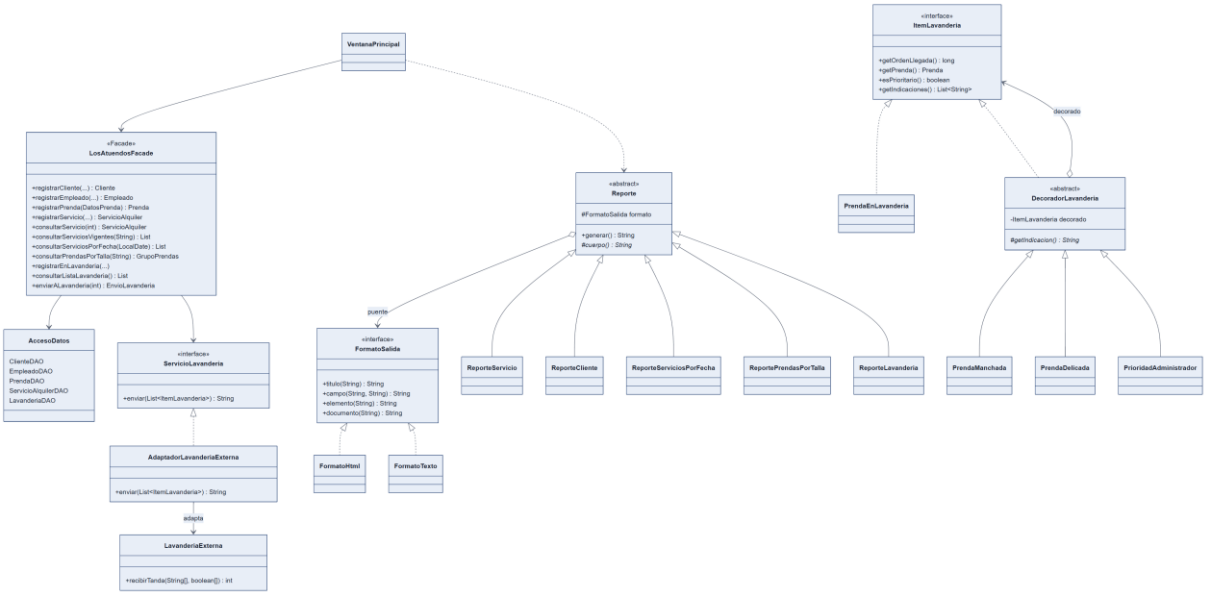


Ilustración 2. *Diagrama de clases de la capa de servicios con Facade, Adapter, Decorator y Bridge. Elaboración propia.*

Diagramas de secuencia por patrón

Singleton

Cuando un DAO necesita la base de datos, pide la instancia de ConexionBD; la primera vez se crea la instancia, se abre la conexión con H2 y se crea el esquema, y las siguientes llamadas reciben la misma conexión.

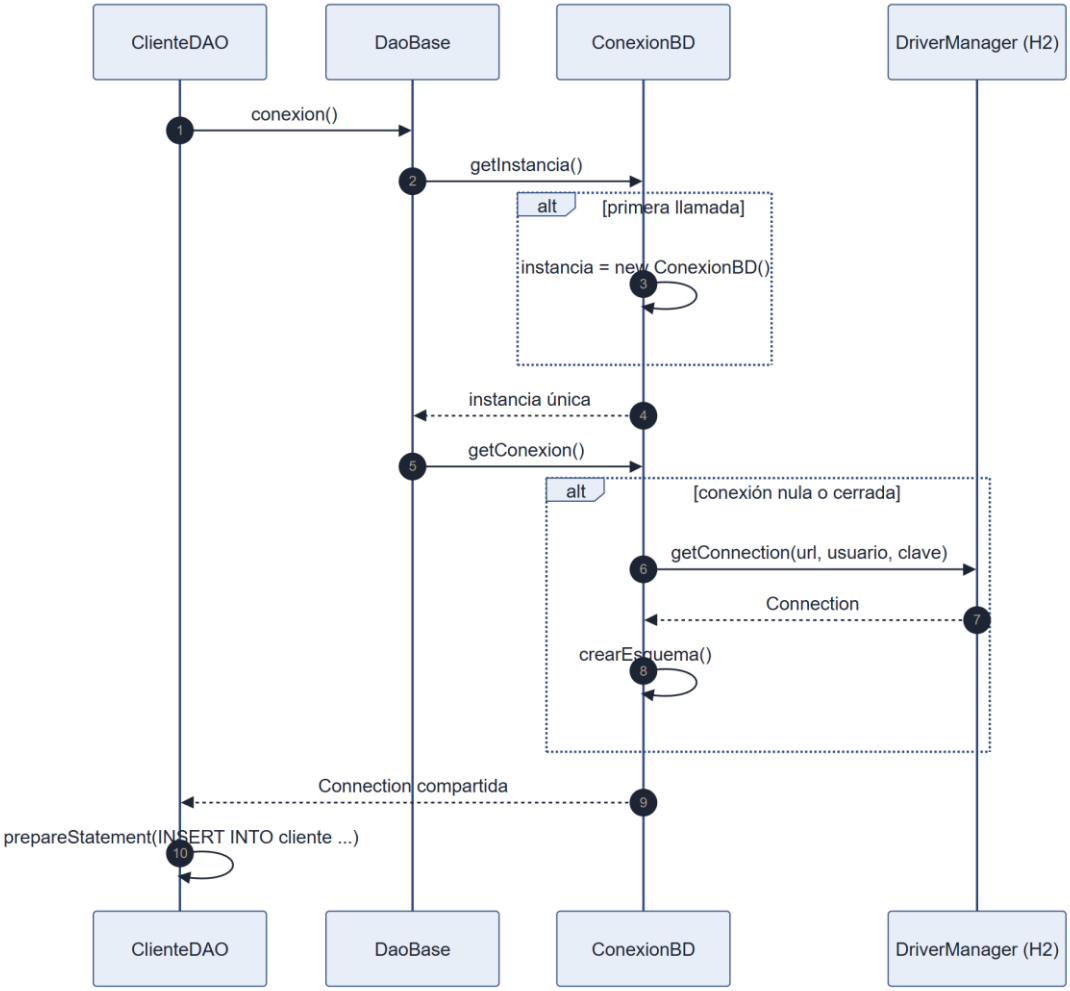


Ilustración 3. *Diagrama de secuencia del patrón Singleton en el caso "Los Atuendos". Elaboración propia.*

Factory Method

Al registrar un traje de caballero, la fachada obtiene la fábrica que corresponde al tipo y le pide crear la prenda; la fábrica valida los datos comunes y su método fábrica instancia la subclase concreta.

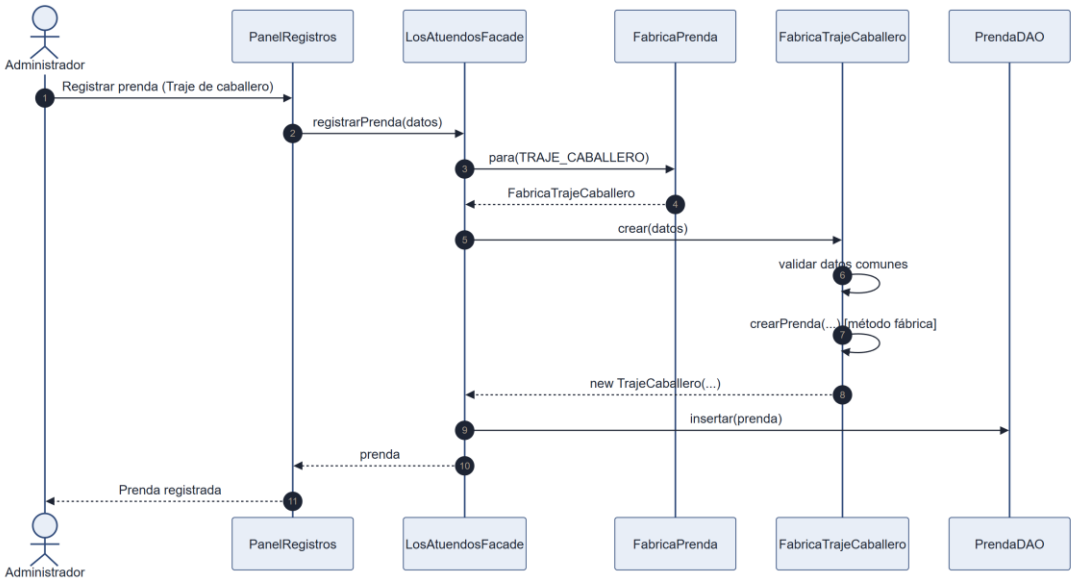


Ilustración 4. *Diagrama de secuencia del patrón Factory Method en el caso "Los Atuendos". Elaboración propia.*

Builder

El registro del servicio arma el objeto paso a paso: fechas, cliente, empleado y cada prenda disponible; al final se asigna el consecutivo y build() verifica que el servicio esté completo antes de guardarlo.

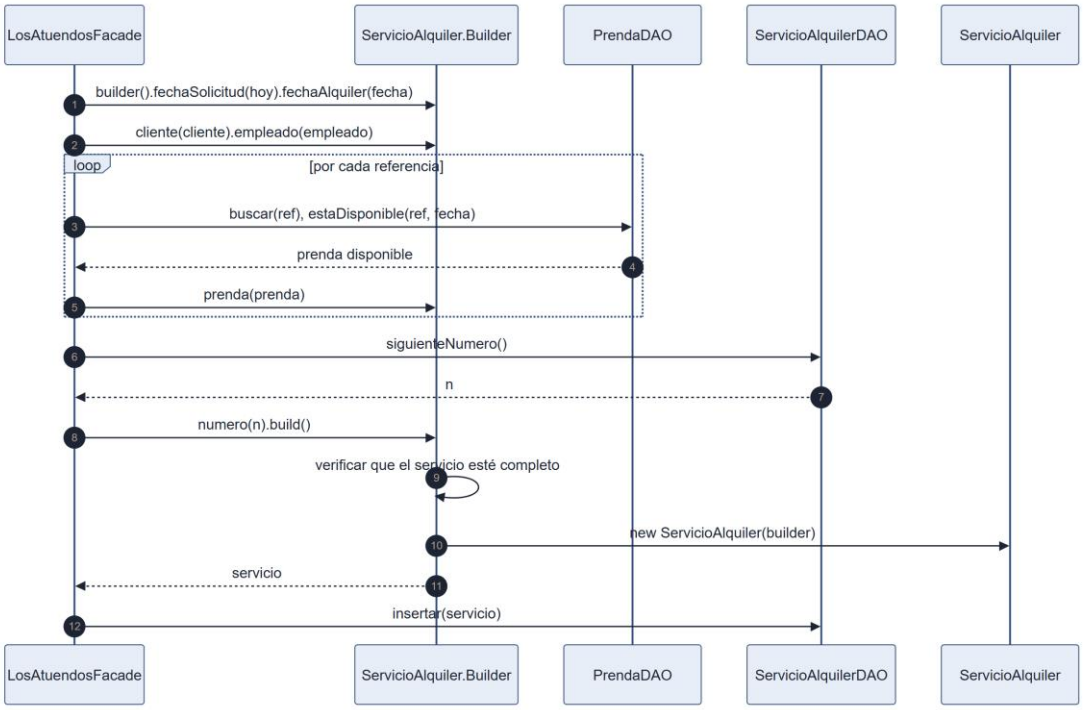


Ilustración 5. *Diagrama de secuencia del patrón Builder en el caso "Los Atuendos". Elaboración propia.*

Composite

En la consulta por talla, la fachada crea un grupo raíz y un grupo por tipo; cuando se pide el valor o las prendas, cada grupo delega en sus hijos sin importar si son prendas u otros grupos.

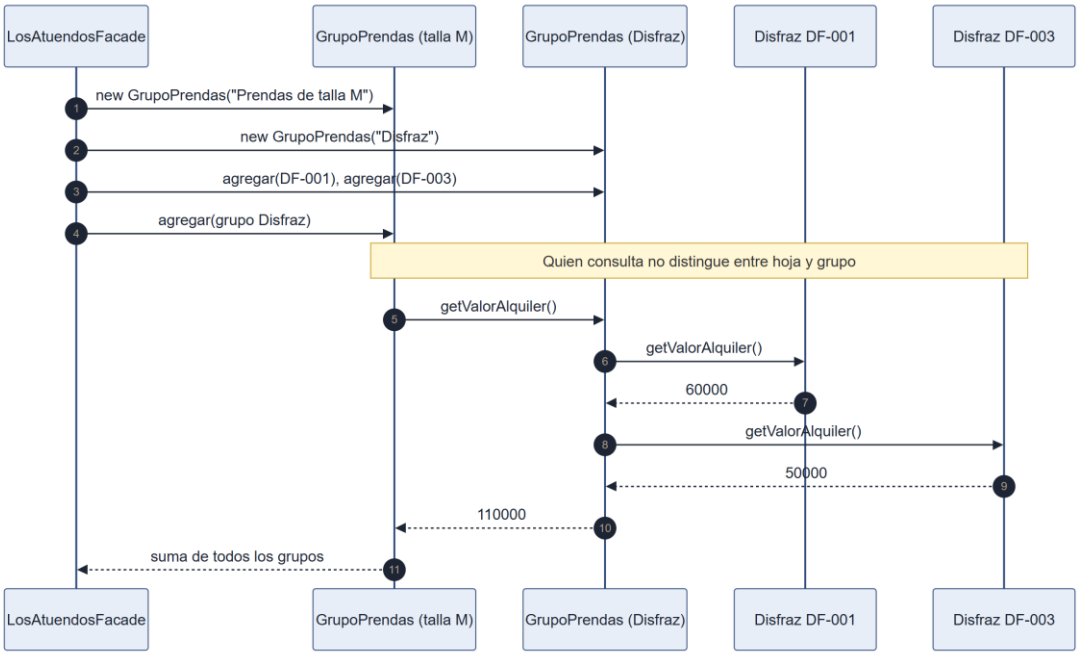


Ilustración 6. *Diagrama de secuencia del patrón Composite en el caso "Los Atuendos". Elaboración propia.*

Facade

El asesor registra un servicio desde la interfaz y la vista hace una sola llamada; la fachada coordina la búsqueda del cliente, del empleado, la disponibilidad de las prendas y el guardado.

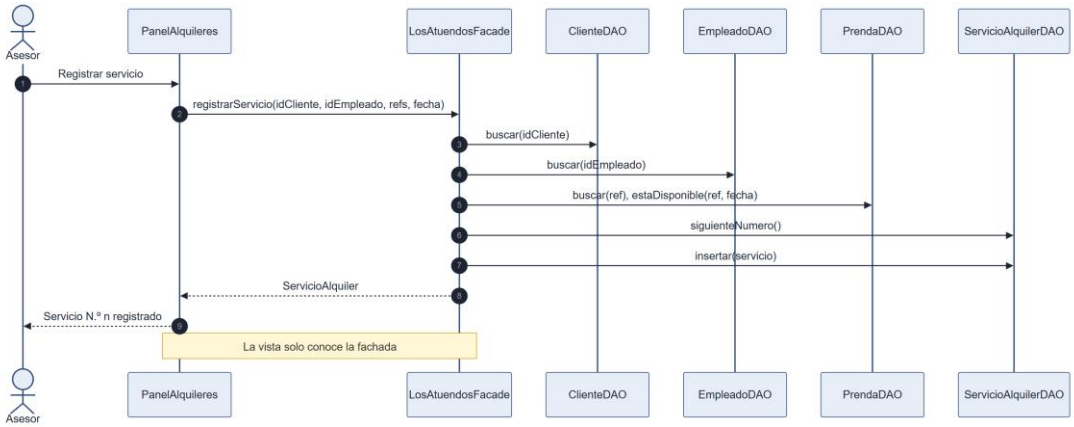


Ilustración 7. *Diagrama de secuencia del patrón Facade en el caso "Los Atuendos". Elaboración propia.*

Adapter

Al enviar una tanda, la fachada entrega la lista de ítems al adaptador, que la convierte a los arreglos que espera el proveedor y traduce el número de tanda en una guía de envío. Es la misma idea que usé en Luks con el conector de WhatsApp: la aplicación habla con su interfaz y el detalle del proveedor queda encerrado.

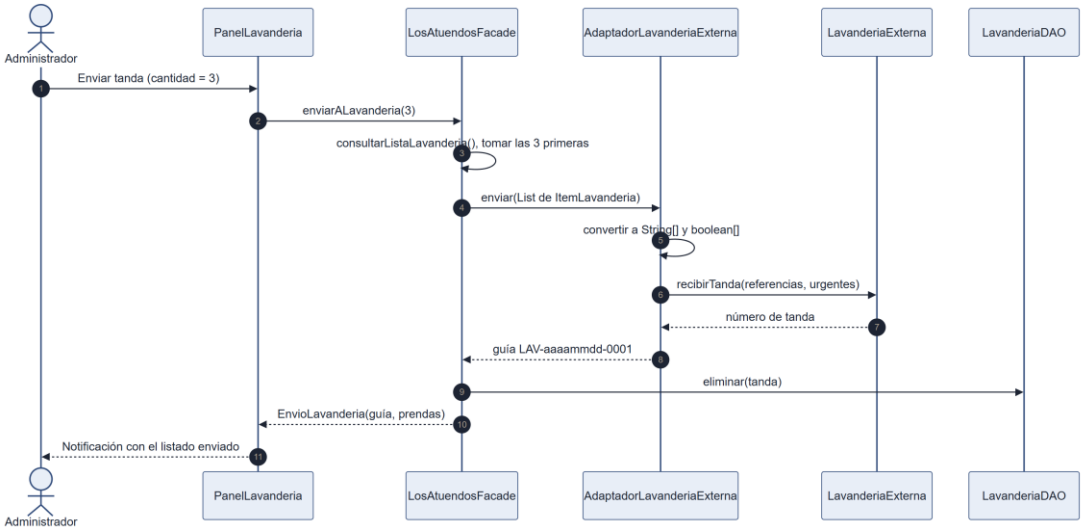


Ilustración 8. *Diagrama de secuencia del patrón Adapter en el caso "Los Atuendos". Elaboración propia.*

Decorator

Una prenda que llegó manchada y es delicada se envuelve dos veces; al pedir las indicaciones, cada decorador agrega la suya a lo que devuelve el objeto que envuelve, y cualquiera de ellos la marca como prioritaria.

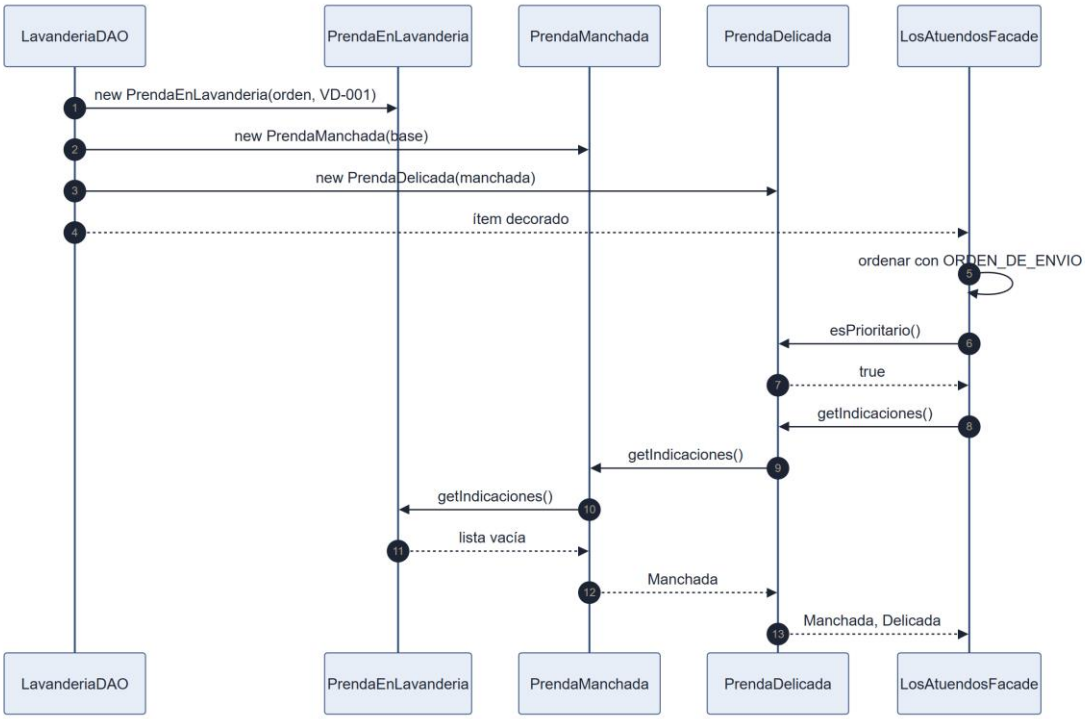


Ilustración 9. *Diagrama de secuencia del patrón Decorator en el caso "Los Atuendos". Elaboración propia.*

Bridge

El mismo ReporteLavanderia se genera con FormatoHtml para el visor de la ventana y con FormatoTexto para la notificación de envío; el reporte decide qué mostrar y el formato decide cómo.

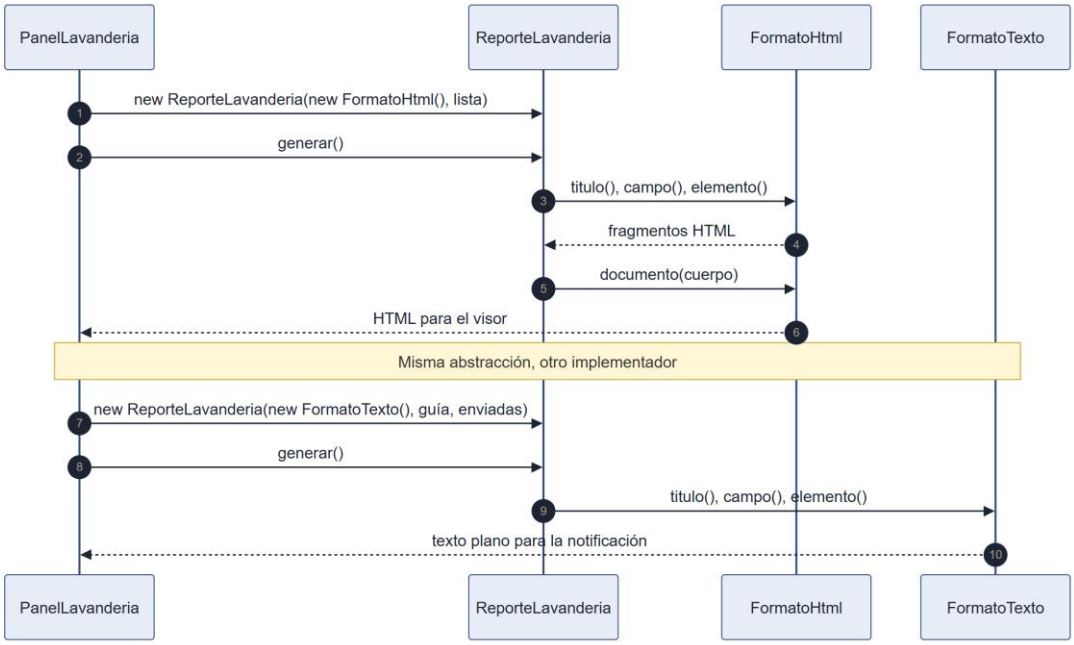


Ilustración 10. *Diagrama de secuencia del patrón Bridge en el caso "Los Atuendos". Elaboración propia.*

Retroalimentación del diseño

El enunciado pide intercambiar el diseño con los compañeros de grupo; como trabajo solo, hice esa revisión sobre mi primera versión y estas fueron las observaciones que cambiaron el diseño:

Observación	Cambio realizado
La prioridad de lavado era un campo booleano en la prenda, y no explicaba por qué la prenda tenía prioridad.	Se reemplazó por el patrón Decorator, cada condición es un decorador que agrega su indicación y da prelación.
La vista armaba el texto de las consultas con concatenaciones, mezclando datos y presentación.	Se agregó el patrón Bridge: reportes por un lado y formatos HTML/texto por el otro.
El servicio de alquiler se podía crear sin prendas.	El Builder valida que el servicio esté completo y la fachada valida existencia y disponibilidad antes de construirlo.
Las prendas leídas de la base de datos se creaban con constructores directos en el DAO.	El DAO también usa las fábricas, así la regla de creación queda en un solo lugar.

Implementación y pruebas

La aplicación está hecha en Java 11 como proyecto de NetBeans (Ant), con interfaz Swing y base de datos relacional H2 en modo archivo, que no necesita instalar un servidor. Los paquetes siguen la responsabilidad de cada parte: *modelo*, *fabrica*, *persistencia*, *lavanderia*, *reporte*, *fachada*, *vista*, *util* y *excepciones*. Las constantes están en la clase Constantes, los atributos son privados y finales cuando no cambian, y todas las operaciones lanzan NegocioException con un mensaje claro, que la ventana muestra sin cerrarse; también hay un manejador global para errores inesperados (Flórez Fernández, 2012; Moreno Pérez, 2015).

El código completo está en el repositorio https://github.com/ChristianMoraLopez/LosAtuendos y en el archivo AA4_Implementación_ChristianReyMoraLopez.zip. Para probar cada funcionalidad escribí la clase PruebaFuncional, que corre contra una base de datos en memoria; la Tabla 2 resume los casos y todos pasaron.

Tabla 2. *Casos de prueba ejecutados*

Funcionalidad	Caso	Resultado
Registro de prendas, clientes y empleados	Prenda válida, prenda sin valor, cliente duplicado, correo inválido	Correcto
Registro de servicio	Consecutivo, prenda ocupada en la fecha, misma prenda otro día, cliente, empleado o prenda no registrados, fecha pasada	Correcto

Consulta de servicio	Datos completos y valor total calculado con el Composite	Correcto
Servicios por cliente	Solo servicios vigentes, ordenados por fecha de alquiler	Correcto
Servicios por fecha	Servicios de una fecha dada	Correcto
Prendas por talla	Prendas separadas por tipo	Correcto
Lavandería	Orden de llegada con prelación, prenda repetida, ver lista sin enviar, envío por tandas, lista vacía	Correcto

Fuente: Elaboración propia

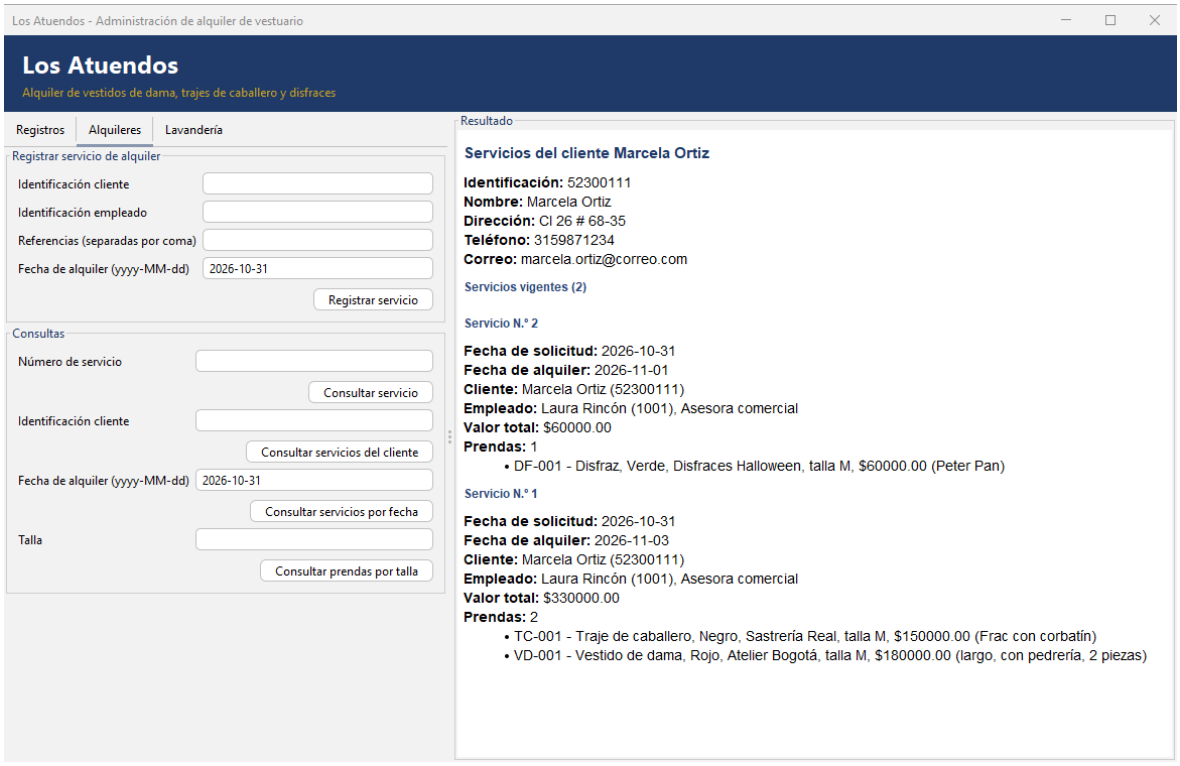


Ilustración 11. *Consulta de servicios vigentes de un cliente en la aplicación. Elaboración propia.*

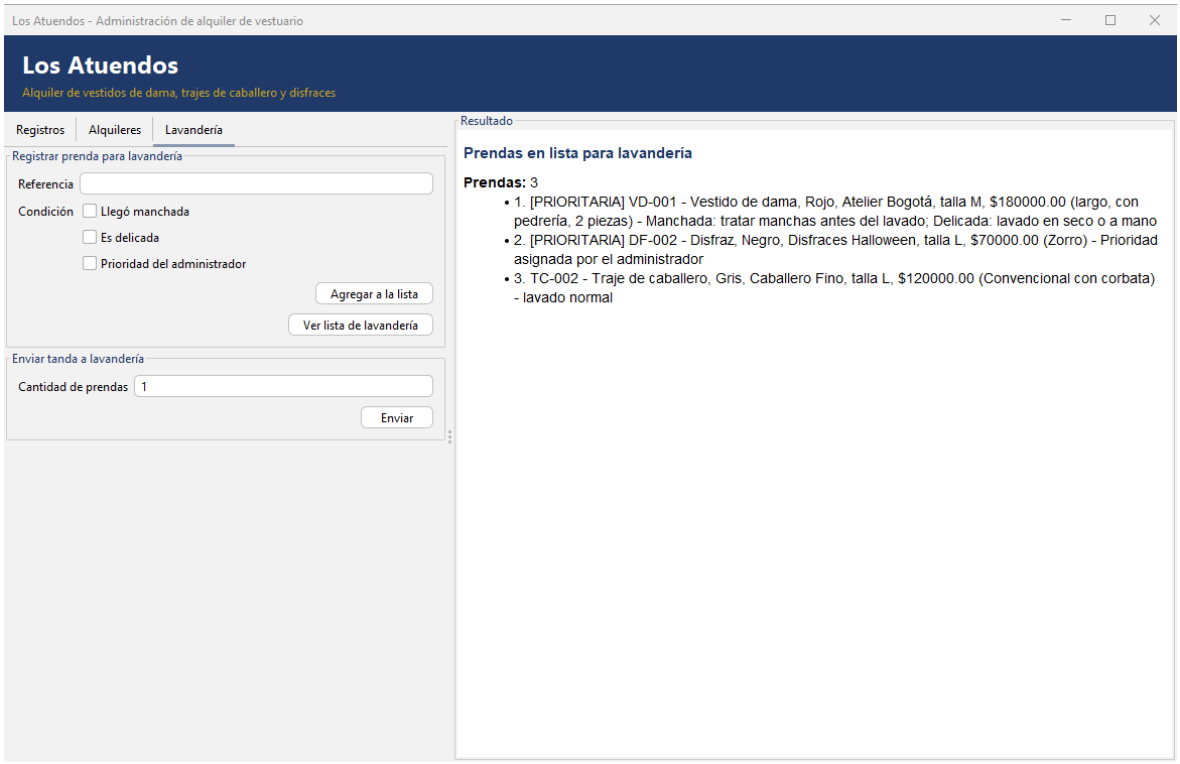


Ilustración 12. *Lista de lavandería con prelación de las prendas decoradas. Elaboración propia.*

```
NOTIFICACIÓN DE ENVÍO A LAVANDERÍA
==================================
Guía de envío: LAV-20261031-0001
Prendas: 2
  * 1. [PRIORITARIA] VD-001 - Vestido de dama, Rojo, Atelier Bogotá, talla M, $180000.00 (largo, con pedrería, 2 piezas) - Manchada: tratar manchas antes del lavado; Delicada: lavado en seco o a mano
  * 2. [PRIORITARIA] DF-002 - Disfraz, Negro, Disfraces Halloween, talla L, $70000.00 (Zorro) - Prioridad asignada por el administrador
```

Ilustración 13. *Notificación de envío a lavandería generada con FormatoTexto. Elaboración propia.*

Conclusiones

Lo que más me queda de esta actividad es que los patrones sirven cuando nacen de un problema real del caso. El Factory Method y el Builder hicieron que crear prendas y servicios fuera seguro, el Composite simplificó los totales y la consulta por talla, y el Decorator resolvió las combinaciones de lavado sin llenar el código de condicionales. La fachada, el adaptador y el puente mantuvieron separadas la vista, las reglas y los proveedores externos, que es justo lo que permite que el sistema siga creciendo.

También entendí que el diseño se revisa antes de programar, la retroalimentación cambió dos decisiones importantes y la implementación salió más corta y más fácil de probar.

Referencias

Besembel, I. (2009). TDSO: una técnica para el diseño de software orientado por objetos. *Revista Ciencia e Ingeniería, 30*(3), 193-200.

Debrauwer, L. (2018). *Patrones de diseño en Java: los 23 modelos de diseño: descripciones y soluciones ilustradas en UML 2 y Java* (4.ª ed.). Ediciones ENI.

Flórez Fernández, H. A. (2012). *Programación orientada a objetos usando Java*. Ecoe Ediciones.

Gamma, E., Helm, R., Johnson, R. y Vlissides, J. (2003). *Patrones de diseño: elementos de software orientado a objetos reutilizable*. Pearson Educación.

Moreno Pérez, J. C. (2015). *Programación orientada a objetos*. RA-MA Editorial.

Anexo A. Código Mermaid de los diagramas

Cada diagrama se puede volver a generar pegando su código en https://mermaid.live o con mermaid-cli (mmdc -i archivo.mmd -o archivo.png). Los archivos .mmd están en la carpeta diagramas del repositorio.

01_clases_dominio_creacionales.mmd

```
        classDiagram
direction TB
class Persona {
  <<abstract>>
  -String identificacion
  -String nombre
  -String direccion
  -String telefono
}
class Cliente {
  -String correo
}
class Empleado {
  -String cargo
}
Persona <|-- Cliente
Persona <|-- Empleado

class ComponenteAlquiler {
  <<interface>>
  +getNombre() String
  +getValorAlquiler() BigDecimal
  +getPrendas() List~Prenda~
}
class Prenda {
  <<abstract>>
  -String referencia
  -String color
  -String marca
  -String talla
  -BigDecimal valorAlquiler
  +getTipo() TipoPrenda*
  +getDetalle() String*
}
class GrupoPrendas {
  -String nombre
  -List~ComponenteAlquiler~ hijos
  +agregar(ComponenteAlquiler)
  +getValorAlquiler() BigDecimal
}
class VestidoDama {
  -boolean pedreria
  -boolean largo
  -int cantidadPiezas
}
class TrajeCaballero {
  -TipoTraje tipoTraje
  -Accesorio accesorio
}
class Disfraz {
  -String nombreDisfraz
}
```

```
ComponenteAlquiler <|.. Prenda
ComponenteAlquiler <|.. GrupoPrendas
GrupoPrendas o--> "0..*" ComponenteAlquiler : hijos
Prenda <|-- VestidoDama
Prenda <|-- TrajeCaballero
Prenda <|-- Disfraz

class ServicioAlquiler {
  -int numero
  -LocalDate fechaSolicitud
  -LocalDate fechaAlquiler
  +builder()$ Builder
  +getValorTotal() BigDecimal
}
class Builder["ServicioAlquiler.Builder"] {
  +numero(int) Builder
  +fechaAlquiler(LocalDate) Builder
  +cliente(Cliente) Builder
  +empleado(Empleado) Builder
  +prenda(Prenda) Builder
  +build() ServicioAlquiler
}
ServicioAlquiler ..> Builder : builder()
Builder ..> ServicioAlquiler : crea
ServicioAlquiler --> "1" Cliente
ServicioAlquiler --> "1" Empleado
ServicioAlquiler *--> "1" GrupoPrendas : prendas

class FabricaPrenda {
  <<abstract>>
  +para(TipoPrenda)$ FabricaPrenda
  +crear(DatosPrenda) Prenda
  #crearPrenda(...) Prenda*
}
class FabricaVestidoDama
class FabricaTrajeCaballero
class FabricaDisfraz
FabricaPrenda <|-- FabricaVestidoDama
FabricaPrenda <|-- FabricaTrajeCaballero
FabricaPrenda <|-- FabricaDisfraz
FabricaVestidoDama ..> VestidoDama : crea
FabricaTrajeCaballero ..> TrajeCaballero : crea
FabricaDisfraz ..> Disfraz : crea

class ConexionBD {
  <<Singleton>>
  -ConexionBD instancia$
  -Connection conexion
  -ConexionBD()
  +getInstancia()$ ConexionBD
  +getConexion() Connection
}
```

02_clases_estructurales.mmd

```
        classDiagram
direction TB
class LosAtuendosFacade {
  <<Facade>>
  +registrarCliente(...) Cliente
  +registrarEmpleado(...) Empleado
  +registrarPrenda(DatosPrenda) Prenda
  +registrarServicio(...) ServicioAlquiler
  +consultarServicio(int) ServicioAlquiler
  +consultarServiciosVigentes(String) List
```

```
  +consultarServiciosPorFecha(LocalDate) List
  +consultarPrendasPorTalla(String) GrupoPrendas
  +registrarEnLavanderia(...)
  +consultarListaLavanderia() List
  +enviarALavanderia(int) EnvioLavanderia
}
class VentanaPrincipal
class AccesoDatos {
  ClienteDAO
  EmpleadoDAO
  PrendaDAO
  ServicioAlquilerDAO
  LavanderiaDAO
}
VentanaPrincipal --> LosAtuendosFacade
LosAtuendosFacade --> AccesoDatos
LosAtuendosFacade --> ServicioLavanderia

class ServicioLavanderia {
  <<interface>>
  +enviar(List~ItemLavanderia~) String
}
class AdaptadorLavanderiaExterna {
  +enviar(List~ItemLavanderia~) String
}
class LavanderiaExterna {
  +recibirTanda(String[], boolean[]) int
}
ServicioLavanderia <|.. AdaptadorLavanderiaExterna
AdaptadorLavanderiaExterna --> LavanderiaExterna : adapta

class ItemLavanderia {
  <<interface>>
  +getOrdenLlegada() long
  +getPrenda() Prenda
  +esPrioritario() boolean
  +getIndicaciones() List~String~
}
class PrendaEnLavanderia
class DecoradorLavanderia {
  <<abstract>>
  -ItemLavanderia decorado
  #getIndicacion() String*
}
class PrendaManchada
class PrendaDelicada
class PrioridadAdministrador
ItemLavanderia <|.. PrendaEnLavanderia
ItemLavanderia <|.. DecoradorLavanderia
DecoradorLavanderia o--> ItemLavanderia : decorado
DecoradorLavanderia <|-- PrendaManchada
DecoradorLavanderia <|-- PrendaDelicada
DecoradorLavanderia <|-- PrioridadAdministrador

class Reporte {
  <<abstract>>
  #FormatoSalida formato
  +generar() String
  #cuerpo() String*
}
class FormatoSalida {
  <<interface>>
  +titulo(String) String
  +campo(String, String) String
  +elemento(String) String
  +documento(String) String
```

```
}
class FormatoHtml
class FormatoTexto
class ReporteServicio
class ReporteCliente
class ReporteServiciosPorFecha
class ReportePrendasPorTalla
class ReporteLavanderia
Reporte o--> FormatoSalida : puente
FormatoSalida <|.. FormatoHtml
FormatoSalida <|.. FormatoTexto
Reporte <|-- ReporteServicio
Reporte <|-- ReporteCliente
Reporte <|-- ReporteServiciosPorFecha
Reporte <|-- ReportePrendasPorTalla
Reporte <|-- ReporteLavanderia
VentanaPrincipal ..> Reporte
```

03_seq_singleton.mmd

```
        sequenceDiagram
autonumber
participant D as ClienteDAO
participant B as DaoBase
participant S as ConexionBD
participant H as DriverManager (H2)
D->>B: conexion()
B->>S: getInstancia()
alt primera llamada
  S->>S: instancia = new ConexionBD()
end
S-->>B: instancia única
B->>S: getConexion()
alt conexión nula o cerrada
  S->>H: getConnection(url, usuario, clave)
  H-->>S: Connection
  S->>S: crearEsquema()
end
S-->>D: Connection compartida
D->>D: prepareStatement(INSERT INTO cliente ...)
```

04_seq_factory_method.mmd

```
        sequenceDiagram
autonumber
actor U as Administrador
participant V as PanelRegistros
participant F as LosAtuendosFacade
participant FP as FabricaPrenda
participant FT as FabricaTrajeCaballero
participant P as PrendaDAO
U->>V: Registrar prenda (Traje de caballero)
V->>F: registrarPrenda(datos)
F->>FP: para(TRAJE_CABALLERO)
FP-->>F: FabricaTrajeCaballero
F->>FT: crear(datos)
FT->>FT: validar datos comunes
FT->>FT: crearPrenda(...) [método fábrica]
FT-->>F: new TrajeCaballero(...)
F->>P: insertar(prenda)
F-->>V: prenda
V-->>U: Prenda registrada
```

05_seq_builder.mmd

```
        sequenceDiagram
autonumber
participant F as LosAtuendosFacade
participant B as ServicioAlquiler.Builder
participant P as PrendaDAO
participant SD as ServicioAlquilerDAO
participant S as ServicioAlquiler
F->>B: builder().fechaSolicitud(hoy).fechaAlquiler(fecha)
F->>B: cliente(cliente).empleado(empleado)
loop por cada referencia
  F->>P: buscar(ref), estaDisponible(ref, fecha)
  P-->>F: prenda disponible
  F->>B: prenda(prenda)
end
F->>SD: siguienteNumero()
SD-->>F: n
F->>B: numero(n).build()
B->>B: verificar que el servicio esté completo
B->>S: new ServicioAlquiler(builder)
B-->>F: servicio
F->>SD: insertar(servicio)
```

06_seq_composite.mmd

```
        sequenceDiagram
autonumber
participant F as LosAtuendosFacade
participant R as GrupoPrendas (talla M)
participant G as GrupoPrendas (Disfraz)
participant P1 as Disfraz DF-001
participant P2 as Disfraz DF-003
F->>R: new GrupoPrendas("Prendas de talla M")
F->>G: new GrupoPrendas("Disfraz")
F->>G: agregar(DF-001), agregar(DF-003)
F->>R: agregar(grupo Disfraz)
Note over R,P2: Quien consulta no distingue entre hoja y grupo
R->>G: getValorAlquiler()
G->>P1: getValorAlquiler()
P1-->>G: 60000
G->>P2: getValorAlquiler()
P2-->>G: 50000
G-->>R: 110000
R-->>F: suma de todos los grupos
```

07_seq_facade.mmd

```
        sequenceDiagram
autonumber
actor U as Asesor
participant V as PanelAlquileres
participant F as LosAtuendosFacade
participant C as ClienteDAO
participant E as EmpleadoDAO
participant P as PrendaDAO
participant S as ServicioAlquilerDAO
U->>V: Registrar servicio
V->>F: registrarServicio(idCliente, idEmpleado, refs, fecha)
F->>C: buscar(idCliente)
F->>E: buscar(idEmpleado)
F->>P: buscar(ref), estaDisponible(ref, fecha)
```

```
F->>S: siguienteNumero()
F->>S: insertar(servicio)
F-->>V: ServicioAlquiler
V-->>U: Servicio N.º n registrado
Note over V,F: La vista solo conoce la fachada
```

08_seq_adapter.mmd

```
        sequenceDiagram
autonumber
actor U as Administrador
participant V as PanelLavanderia
participant F as LosAtuendosFacade
participant A as AdaptadorLavanderiaExterna
participant L as LavanderiaExterna
participant D as LavanderiaDAO
U->>V: Enviar tanda (cantidad = 3)
V->>F: enviarALavanderia(3)
F->>F: consultarListaLavanderia(), tomar las 3 primeras
F->>A: enviar(List de ItemLavanderia)
A->>A: convertir a String[] y boolean[]
A->>L: recibirTanda(referencias, urgentes)
L-->>A: número de tanda
A-->>F: guía LAV-aaaammdd-0001
F->>D: eliminar(tanda)
F-->>V: EnvioLavanderia(guía, prendas)
V-->>U: Notificación con el listado enviado
```

09_seq_decorator.mmd

```
        sequenceDiagram
autonumber
participant D as LavanderiaDAO
participant B as PrendaEnLavanderia
participant M as PrendaManchada
participant DL as PrendaDelicada
participant F as LosAtuendosFacade
D->>B: new PrendaEnLavanderia(orden, VD-001)
D->>M: new PrendaManchada(base)
D->>DL: new PrendaDelicada(manchada)
D-->>F: ítem decorado
F->>F: ordenar con ORDEN_DE_ENVIO
F->>DL: esPrioritario()
DL-->>F: true
F->>DL: getIndicaciones()
DL->>M: getIndicaciones()
M->>B: getIndicaciones()
B-->>M: lista vacía
M-->>DL: Manchada
DL-->>F: Manchada, Delicada
```

10_seq_bridge.mmd

```
        sequenceDiagram
autonumber
participant V as PanelLavanderia
participant R as ReporteLavanderia
participant H as FormatoHtml
participant T as FormatoTexto
V->>R: new ReporteLavanderia(new FormatoHtml(), lista)
V->>R: generar()
R->>H: titulo(), campo(), elemento()
```

```
H-->>R: fragmentos HTML
R->>H: documento(cuerpo)
H-->>V: HTML para el visor
Note over V,T: Misma abstracción, otro implementador
V->>R: new ReporteLavanderia(new FormatoTexto(), guía, enviadas)
V->>R: generar()
R->>T: titulo(), campo(), elemento()
T-->>V: texto plano para la notificación
```